

CSCI4390-6390 Assign1

Assign1

Due Date: Sep 15th (Fri), before midnight (11:59:59PM EDT)

Download the [Wine Quality Dataset](#) from the UCI Machine Learning repository. Extract the winequality-red.csv datafile that records 12 attributes about 1599 instances of red wine. You should parse and store the data as a data matrix, after ignoring the last "quality" attribute, which is an integer valued target variable. Thus, your data matrix will have 1599 points with 11 dimensions.

You must submit a self-contained jupyter notebook, with all of your **code and output**. You must use NumPy, with well known/inbuilt libraries for data input (e.g., pandas). Plots must be in inline mode (i.e., embedded) in the notebook, using matplotlib.

Part I: Basic Stats (50 Points)

a. Mean

Compute the sample mean vector $\boldsymbol{\mu}$ for the data matrix using the column view in $\mathbf{R}^n$. That is, project each attribute vector $\mathbf{X}_j$ onto the ones vector $\mathbf{1}$. Verify the answer using the numpy mean function.

b. Variance

Compute the sample variance for each attribute σ_j^2 (for all $j = 1, 2, \dots, d$) using the column view. That is, use the avg. squared norm of the centered attribute vectors $\tilde{\mathbf{X}}_j$. Verify the answers using the numpy var function.

Next, compute the total variance $\text{var}(\mathbf{D})$ (see Eq. (1.8)), and verify that the sum of all the attribute variances equals the total variance.

c. Covariance matrix

Compute the sample covariance matrix $\boldsymbol{\Sigma}$ as **inner products** between the attributes of the centered data matrix (see Eq. (2.38) in chapter 2). This is the column view.

Next, compute the sample covariance matrix as sum of the **outer products** between the centered points (see Eq. (2.39)). This is the row view.

Print the matrix from both the approaches. You can verify your output using numpy cov function (make sure to set rowvar as False and bias as True).

d. Correlation matrix

Compute the correlation matrix for this dataset using the formula for the cosine between centered attribute vectors (see Eq. (2.30)).

Output which attribute pairs are i) the most correlated, ii) the most anti-correlated, and iii) the least correlated or uncorrelated? Create the scatter plots for these three interesting pairs and visually confirm the trends, i.e., describe how each of the three cases results in a particular type of plot.

Part II: Mean via Gradient Descent (50 points)

In class, we saw how the mean is the point that minimizes the sum of squared distance to all the points in the dataset. Here, we will find the mean via stochastic gradient descent, i.e., updating the estimate using the gradient information from a single random point each time (or a mini-batch of points).

(CSCI 4390 Only) Stochastic Gradient Descent (SGD)

Assume we want to find the point $\mathbf{z}$ that optimizes the squared distance objective, but based on a single point $\mathbf{x}_i$. The gradient of the objective

$$J = \frac{1}{2} \|\mathbf{x}_i - \mathbf{z}\|^2$$

is given as

$$\nabla = \frac{\partial J}{\partial \mathbf{z}} = \mathbf{z} - \mathbf{x}_i$$

To get to the minimum point, we have to update the current estimate of $\mathbf{z}$ by taking a small step $\eta > 0$ in a direction opposite to the gradient ∇ , given as

$$\mathbf{z} = \mathbf{z} - \eta \cdot \nabla$$

Or,

$$\mathbf{z} = \mathbf{z} - \eta(\mathbf{z} - \mathbf{x}_i)$$

So, starting from a random $\mathbf{z}$ vector, we can repeatedly sample a single point from the data matrix (via, say, numpy random.choice function), and update $\mathbf{z}$ using the equation above.

To test for convergence, we compute the norm of the difference between the previous value of $\mathbf{z}$ and its updated value, and stop if the difference falls below a threshold ϵ .

Implement this SGD algorithm, and show how close is the gradient descent estimate of the mean from the actual sample mean. Show the estimated mean, and the norm of its difference from the actual mean.

To make this work you have to carefully choose the values of the step size η and the convergence threshold ϵ . Note that if you make η too small then you do not get good estimates, and if you make ϵ too small you may never converge. You have to find a balance.

(CSCI 6390 Only) Mini-batch Stochastic Gradient Descent (SGD)

You will implement the SGD, but using a mini-batch of points, rather than a single point. Assume you sample a batch of B points (say, $B = 100$) from the data (can be done using the numpy random.choice function). Then the objective is given as

$$J = \frac{1}{2} \sum_{i=1}^B \|\mathbf{x}_i - \mathbf{z}\|^2$$

And the gradient is given as

$$\nabla = B \cdot \mathbf{z} - \sum_{i=1}^B \mathbf{x}_i$$

So, starting from a random estimate of $\mathbf{z}$ you can use batches of B random points to update the estimate, using the same gradient descent equation, but adjusted to reflect that now you are working with B points, and adjusting η accordingly:

$$\mathbf{z} = \mathbf{z} - \frac{\eta}{B} \cdot \nabla$$

Or

$$\mathbf{z} = \mathbf{z} - \eta \cdot \mathbf{z} + \frac{\eta}{B} \sum_{i=1}^B \mathbf{x}_i$$

To test for convergence, we compute the norm of the difference between the previous value of $\mathbf{z}$ and its updated value, and stop if the difference falls below a threshold ϵ .

Implement this algorithm, and show the estimated solution, as well as the difference (norm) from the sample mean. To make this work you have to carefully choose the values of the step size η and the convergence threshold ϵ .

Part III: Eigenvectors and Principal Components (50 Points)

a. (CSCI-4390 Only): Dominant Eigenvector and Eigenvalue

Compute the dominant eigenvalue and eigenvector of the covariance matrix Σ via the power-iteration method, as described below.

Start from a random vector $\mathbf{x}_0 \in \mathbf{R}^d$, where d is the number of dimensions, and normalize it to be a unit vector. Set $t = 0$ (t denotes the iteration number).

Next compute the updated vector by left multiplication with the covariance matrix:

$$\mathbf{x}_{t+1} = \Sigma \mathbf{x}_t$$

Record the ratio of the value of an element in the updated vector with respect to the previous vector ((for, say, the max valued element and same index) -- this is the estimate for the eigenvalue. After computing the ratio, normalize $\mathbf{x}_{t+1}$ to be a unit vector, increment t , and repeat this process.

To test convergence, compute the norm of the difference between the previous and updated vector (before normalization step), and stop if this norm falls below some threshold ϵ .

Print out $\mathbf{u}_1$ and λ_1 . You can verify your answer using numpy linalg.eigh function.

Once you have obtained the dominant eigenvector, $\mathbf{u}_1$ project each of the original data points $\mathbf{x}_i$ onto this vector, and plot the coordinates for the new points along this principal component "direction".

Separately, plot your estimate for the eigenvalue λ_1 as a function of t .

Finally, compute the variance of the projected points along $\mathbf{u}_1$. What is its relationship to λ_1 .

b. (CSCI-6390 Only) First Two Eigenvectors and Eigenvalues

Compute the first two eigenvectors of the covariance matrix Σ using a generalization of the power-iteration method.

Let $\mathbf{X}_0$ be a $d \times 2$ (random) matrix with two non-zero d -dimensional column vectors, and let $\mathbf{X}_t$ be the current estimate, where t is the step number. We will iteratively orthogonalize, normalize, and left multiply with Σ . That is, first orthogonalize the second column with respect to the first one by subtracting its projection along the first column (see section 1.3.3 in chapter 1). After orthogonalizing, normalize both columns to be unit length, and then do the left multiplication

$$\mathbf{X}_{t+1} = \Sigma \mathbf{X}_t$$

To get the eigenvalues, compute the ratio of the values in both columns after to before update (for, say, the max valued element and same index). These denote the current estimates for each of the corresponding eigenvalues, λ_1 and λ_2 . After computing the ratios, orthogonalize and normalize the vectors, and repeat the above steps until convergence.

To test for convergence, compute the Frobenius norm between $\mathbf{X}_{t+1}$ and $\mathbf{X}_t$. If the difference is less than some threshold ϵ then we stop. Both matrices are assumed to be orthogonalized and normalized.

Once you have obtained the two eigenvectors $\mathbf{u}_1$ and $\mathbf{u}_2$, (scalar) project each of the original data points $\mathbf{x}_i$ onto those two vectors, and plot these projected points in the two new dimensions.

Print out $\mathbf{u}_1$ and $\mathbf{u}_2$, and λ_1 and λ_2 . You can verify your answer using numpy linalg.eigh function.

Separately, plot the graphs for the estimates of λ_1 and λ_2 as a function of t .

Finally, compute the variance of the projected points along $\mathbf{u}_1$ and $\mathbf{u}_2$. What is their relationship to λ_1 and λ_2 .

Submission

Submit your notebook via submittly, named **assign1.ipynb**. The notebook should be self-contained, i.e., it should include all output from all the parts, including figures. It should not hardcode file paths, but rather assume that the csv datafile is in the current directory, so only the csv filename should be used. Do not submit the datafile.

If you decide to consult ChatGPT (or other similar AI tools), you must declare this, and submit a separate text/doc/PDF file documenting your successes and failures, e.g., what prompts you tried, what worked, what did not work, how you fixed it, etc. Name this file **declaration.[txt/pdf/doc]**, choosing the correct extension.

For those not that familiar with python or NumPy, you may search online for tutorials, e.g. [Python tutorial](#) or [NumPy](#).

Policy on Academic Honesty

You are free to discuss how to tackle the assignment, but all coding must be your own. Any AI tool use must be declared. Any students caught violating the academic honesty principle (e.g., code similarity, or failure to disclose AI tools) will get an automatic F grade on the course and will be referred to the dean of students for disciplinary action.

Contents © 2023 [Mohammed J. Zaki](#) - Powered by [Nikola](#)